

OpenClaw AI guessing game tutorial

Before starting, please complete [01-Peripheral hardware configuration], [02-openclaw API-KEY configuration], [03-openclaw switching model], [04-OpenClaw prompt words], [05-AI voice interaction configuration], [06-Three dialogue scheme configuration tests] in the `Preparation before use` directory. If you are currently only using TUI/WhatsApp, you can skip 05 first. This article assumes that the basic environment has been prepared and only the AI guessing project itself will be launched.

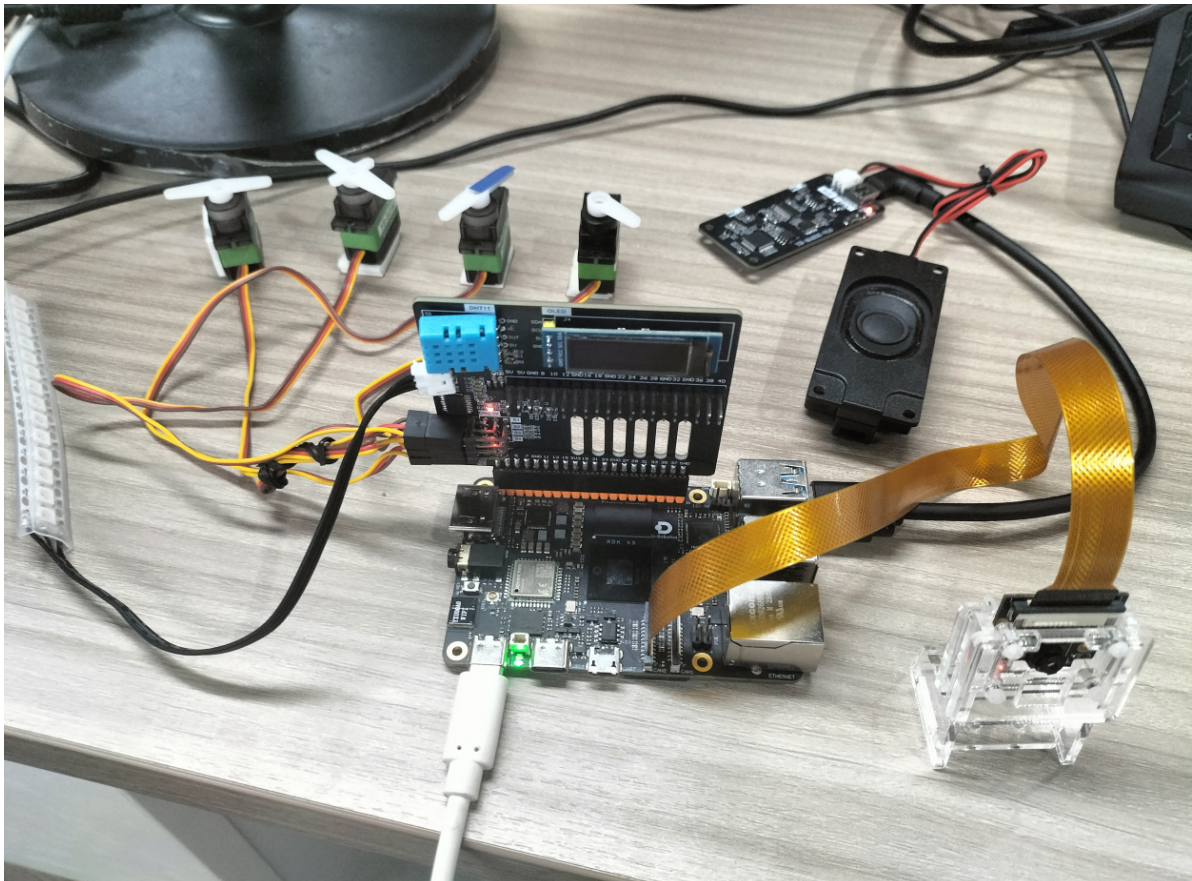
1. Tutorial Objectives

This article demonstrates OpenClaw's interactive gaming capabilities through "gesture recognition + AI referee + screen light feedback". After completing this tutorial, readers will be able to:

- Learn how to use the camera to recognize player gestures
- Converse with OpenClaw through WhatsApp, TUI, and voice code
- Complete four project scenarios: gesture recognition, AI punching, result determination, and scoring feedback

2. Hardware preparation

- OpenClaw main control board
- CSI camera module
- OLED display
- RGB light strip/RGB lamp bead module
- [Optional] AI voice module + speaker
- Wiring diagram



3. Software preparation

Code entry that will be used in the tutorial

- Camera photo command: `/home/sunrise/hardware_hal_demo/camera`
- Visual analysis script: `/home/sunrise/.openclaw/workspace/skills/sunrise-vision/scripts/csi_describe`
- Voice entrance: `/home/sunrise/voice_to_openclaw/src/main.py`

The key capabilities of this project mainly include three sections:

- The camera is responsible for recording player gestures
- `csi_describe` gesture is responsible for recognizing gestures
- OpenClaw is responsible for deciding the AI punch and displaying the results to OLED / RGB

4. Test the basic capabilities of the project

Before starting the current project, please complete a basic dialogue self-test of `openclaw tui`; if you have not done so yet, please read [06-Three dialogue scheme configuration tests] first. After passing, use TUI to verify the basic capabilities in the project.

Terminal input:

```
openclaw tui
```

```
sunrise@ubuntu:~/openclaw$ openclaw tui
♥ OpenClaw 2026.3.24 (cfff6dc9) - You had me at 'openclaw gateway start.'
openclaw tui - ws://127.0.0.1:18789 - agent main - session main
session agent:main:main

hello

你好呀，主人！☺ 我是小亚，随时待命。

有什么需要我帮忙的吗？
gateway connected | idle
agent main | session main (openclaw-tui) | deepseek/deepseek-v4-pro | tokens 12k/1.0m (1%)
```

Then you can do a few basic tests first:

- Take a photo of the gesture for me
- Identify whether this is Scissorhands
- Display "Ready" on OLED
- Make RGB bright blue

5. Three options for talking to OpenClaw

Solution	Suitable scenario	Advantages	Suggested positioning
WhatsApp	Remote demonstration, event interaction	Easy to share with multiple people to experience	Demonstrate "visual interactive game"
TUI	Local debugging, rule verification	Easiest to see the identification and judgment process	Run through the game logic first
Voice code	Game interaction, classroom demonstration	More like a real AI game build	As a complete demonstration solution

If you have selected the AI voice module + speaker, Feishu dialogue, tui, and WhatsApp dialogue need to broadcast the content of openclaw's reply. You can run the following program on the terminal. The AI voice module dialogue does not need to be run repeatedly

- Modify whether to open broadcast parameters

```
/home/sunrise/voice_to_openclaw/.env
```

```
ENABLE_TTS=true #This parameter is to control whether to broadcast. After modification, save it and run the following program on the terminal.
```

- Terminal input

```
cd ~/voice_to_openclaw
source .venv/bin/activate
python -m src.openclaw_session_tts
```

```
(.venv) sunrise@ubuntu:~/voice_to_openclaw$ python -m src.openclaw_session_tts
[系统] OpenClaw 会话播报器已启动
[系统] 监听目录=/home/sunrise/.openclaw/agents/main/sessions
[系统] 监听渠道=feishu,tui,whatsapp
```

5.1 WhatsApp solution

For WhatsApp access and robot creation steps, please refer directly to [06-Configuration Test of Three Dialogue Schemes]. After creating it, you can remotely trigger the guessing game in WhatsApp, for example:

- Let's play jackpot, take a photo and recognize my gestures. You can randomly make gestures and then judge whether you won or I won. The OLED will show who won this round.

20:27



< 2

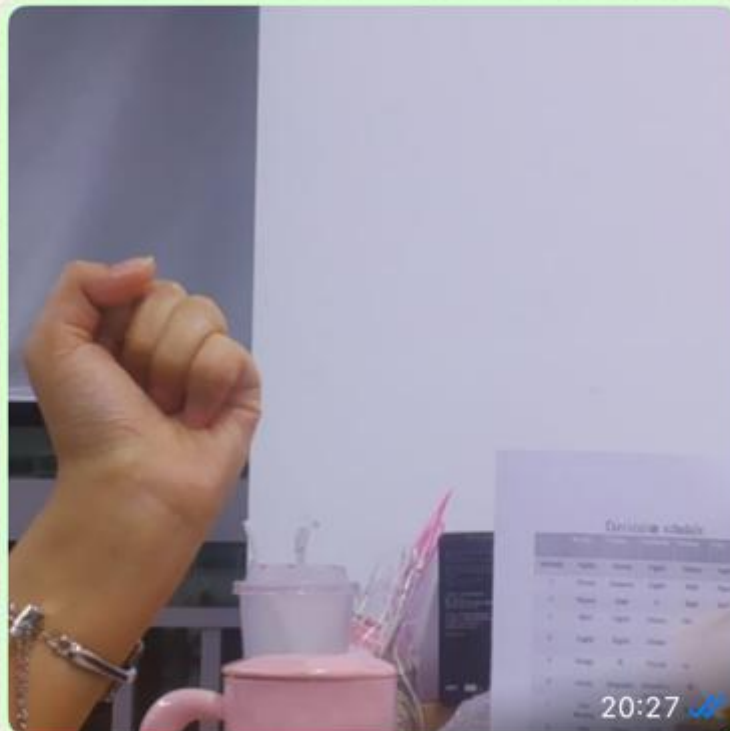


+86 153 3885 7526 (你)

给自己发消息

Please play rock paper scissors.
Take a photo to recognize my
gesture. You can randomly make a
gesture. Then judge whether you win
or I win OLED displays who this round.

20:27 ✓✓



I can see the user's hand gesture in
the image - it's a **fist**, which
represents **ROCK** in rock paper
scissors.

Now I need to:

1. Make a random gesture choice
(rock, paper, or scissors)
2. Determine the winner
3. Display the result on OLED



Here you can add screenshots of WhatsApp conversations by yourself.

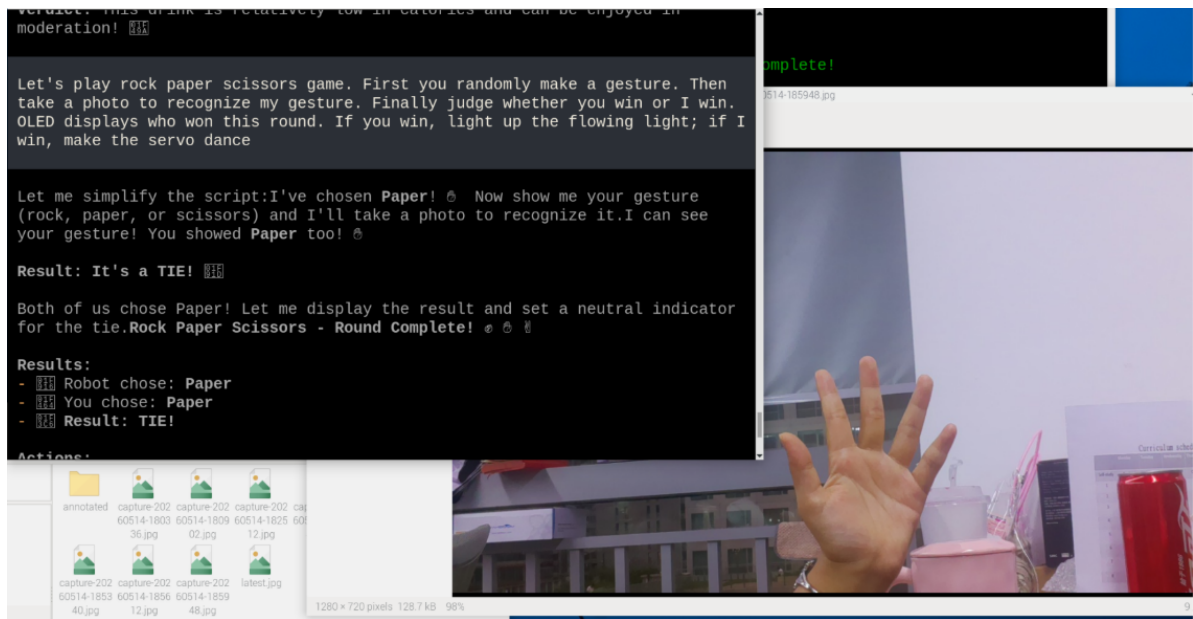
5.2 TUI scheme

If you have completed the TUI self-test in Part 6, enter the terminal text dialogue directly:

```
openclaw tui
```

Then enter project-related instructions, such as:

- Let's play the guessing game. You make random gestures first, then take a photo to recognize my gestures, and finally judge whether you won or I won. The OLED will show who won this round. If you win, the running lights will light up. If I win, the servo will do a dance.

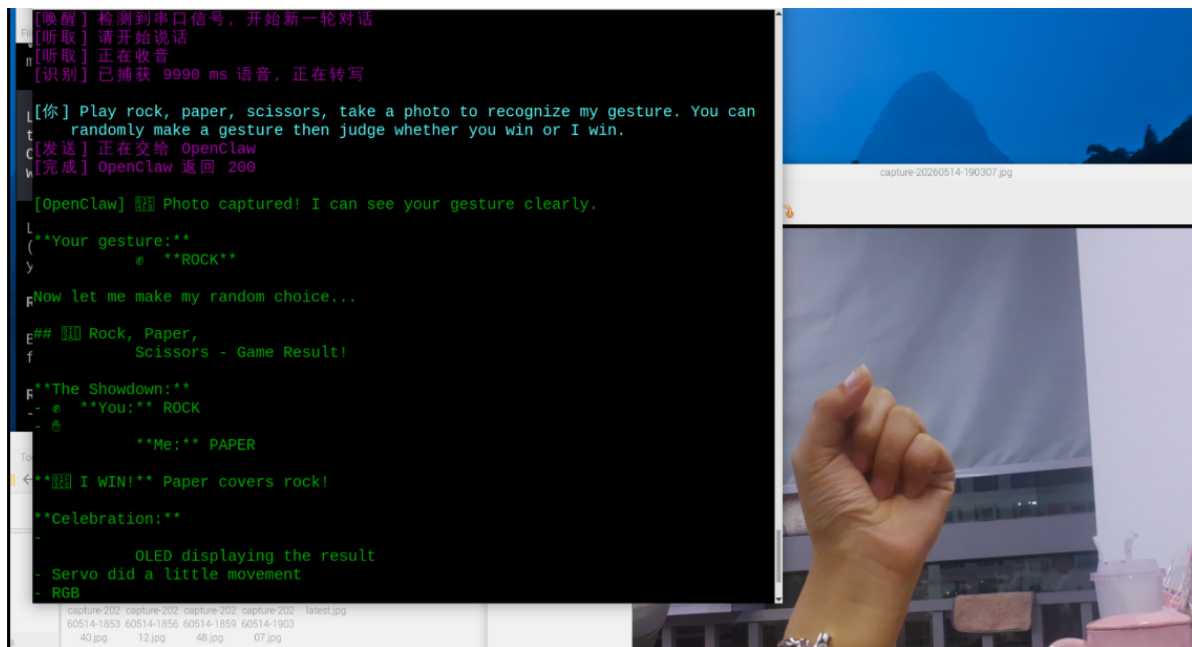


5.3 Voice code scheme

```
Requires optional ai voice interaction module + speaker
```

For voice module installation, environment configuration and basic wake-up process, please refer directly to [05-AI Voice Interaction Configuration]. After waking up, you can directly say the control content, such as:

- Let's play jackpot, take a photo and recognize my gestures. You can randomly make gestures and then judge whether you won or I won. The OLED will show who won this round.



6. Project implementation

Note: The following are all dialogue demonstrations in `openc1aw tui` mode. You can also choose WhatsApp or voice mode to complete it.

6.1 Scenario 1: Gesture recognition

Experimental goals

Recognizes the player's current gesture, such as rock, scissors, or paper.

Effect description

OpenClaw will first take a picture and then identify the gesture type based on the picture. This step is the input basis for the entire game.

Example instructions

- Recognize the gesture I am making now
- I've already punched. Please look at it for me.
- See if it's rock, scissors or paper

6.2 Scenario 2: AI punching and judgment

Experimental goals

After OpenClaw recognizes the player's gestures, it gives the AI's punch results and determines victory or defeat.

Effect description

This link best reflects the characteristics of "visual input + conversational reasoning". OpenClaw not only identifies players, but continues to participate in the game, giving its own punches and interpretations of the results.

Example instructions

- Let's play jackpot, take a photo and recognize my gestures. You can randomly make gestures and then judge whether you won or I won. The OLED will show who won this round.

6.3 Scenario 3: OLED scoreboard

Experimental goals

Let the OLED display the current game result or cumulative score.

Effect description

The screen can display `win`, `Lose`, `Draw`, or a simple score like `You 1 : 0 AI`, making the project more like a complete mini-game.

Example instructions

- Display the result of the round on OLED
- Write the score on the screen
- Display "Round 1 You Win"

6.4 Scenario 4: RGB result feedback

Experimental goals

Use RGB colors to quickly express wins and losses.

Effect description

For example, a win will light green, a loss will light red, and a draw will light blue. In this way, even without reading the text, you can immediately know the result of this round.

Example instructions

- Green light if I win
- If I lose, the red light will turn on.
- If you win the running water lamp, I win the servo and do a dance